**Universidad Peruana Cayetano Heredia**
**Facultad de Ciencias e Ingeniería**
**Departamento de Ciencias Exactas**

# Programación Avanzada 2025

# Lab. 7.2. POO. Herencia de Objetos en Python

***Octubre 03, 2025***

**Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345**

**Herencia y la Relación "es un" ("is a")**

Cuando un objeto es una versión especializada de otro objeto, existe una relación "es un" entre ellos. Por ejemplo, un saltamontes es un insecto. Aquí hay algunos otros ejemplos de la relación "es un":

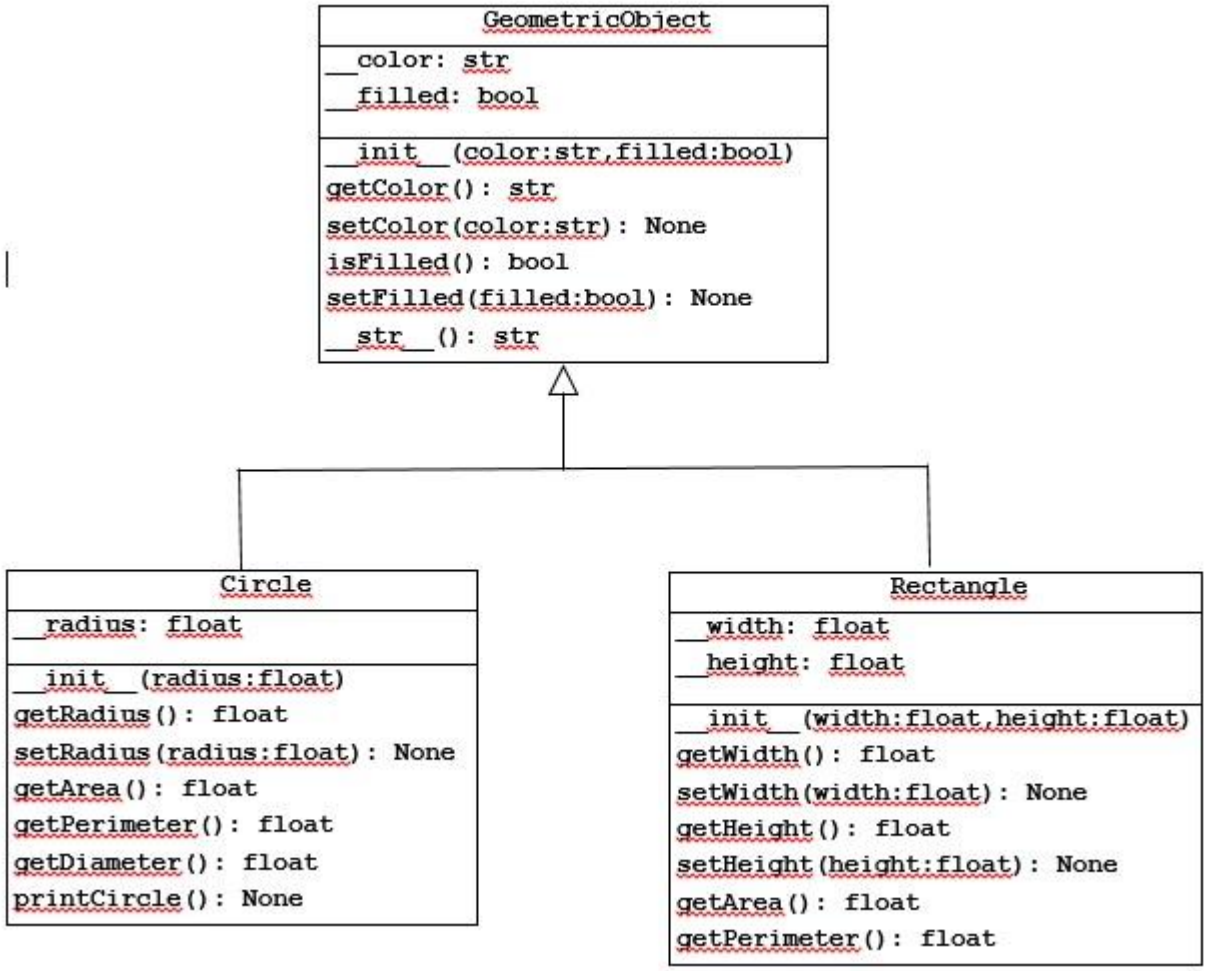
- Un caniche es un perro.
- Un automóvil es un vehículo.
- Una flor es una planta.
- Un rectángulo es una figura geométrica.
- Un jugador de fútbol es un deportista.

Cuando existe una relación "es un" entre objetos, significa que el objeto especializado tiene todas las características del objeto general, más características adicionales que lo hacen especial. En la Programación Orientada a Objetos, Herencia se utiliza para crear una relación "es un" entre clases. Esto permite ampliar las capacidades de una clase creando otra clase que es una versión especializada del mismo.

Herencia implica una superclase y una subclase. La *superclase* es la clase general y la *subclase* es la clase especializada. Se puede pensar en la subclase como una versión extendida de la superclase. La subclase hereda atributos y métodos de la superclase sin que sea necesario reescribir ninguno de ellos. Además, se pueden agregar nuevos atributos y métodos a la subclase, y eso es lo que la convierte en una versión especializada de la superclase.

NOTA: Las superclases también se denominan *clases base* y las subclases también se denominan *clases derivadas*. Cualquiera de los términos es correcto. En el curso usaremos ambos términos, superclase y clase base.

El siguiente diagrama UML muestra la relación de herencia de tres clases.
La clase `GeometricObject` es la clase padre (superclase), modela a un objeto geométrico de forma general, como atributos tiene las características que están presentes en cualquier objeto geométrico, en este caso: `color` y `filled` (relleno). Tiene cinco métodos y un inicializador.
La clase `Circle` y la clase `Rectangle`, son subclases (clases hijo) de `GeometricObject`. Ambas heredan los atributos y métodos de la clase padre, adicionalmente, ambas tienen sus propios atributos y métodos. Es común decir que la clase hijo *extiende* a la clase padre.



1. Escriba una clase `Multimedia` para almacenar información de objetos de tipo multimedia (películas, discos, mp3,…). Esta clase contiene título, autor, formato y duración como atributos. El formato puede ser uno de los siguientes: wav, mp3, midi, avi, mov, mpg, cdAudio y dvd. El valor de todos los atributos se pasa por parámetro en el momento de crear el objeto. Esta clase tiene, además, un método para devolver cada uno de los atributos y un método `__str()__` que devuelve la información del objeto. Por último un método `equals` que recibe un objeto de tipo Multimedia y devuelve `True` en caso de que el título y el autor sean iguales a los del objeto que llama al método y `False` en caso contrario.

   Escriba una clase `Pelicula` que herede de la clase `Multimedia` anterior. La clase `Pelicula` tiene, además de los atributos heredados, un actor principal y una actriz principal. Se permite que uno de los dos sea nulo, pero no los dos. La clase debe tener dos métodos para obtener los dos nuevos atributos y debe sobrescribir el método __str__() para que devuelva, además de la información heredada, la nueva información.

   Escriba una clase `ListaMultimedia` para almacenar objetos de tipo multimedia. La clase debe tener un atributo que sea un array de objetos `Multimedia` y un entero para contar cuántos objetos hay almacenados. Además, tiene un constructor (inicializador) y los siguientes métodos:
   - El constructor recibe por parámetro un entero indicando el número máximo de objetos que va a almacenar.
   - `size()`: devuelve el número de objetos que hay en la lista.
   - `add(Multimedia m)`: añade el objeto al final de la lista y devuelve `True`, en caso de que la lista esté llena, devolverá `False`.
   - `get(int posicion)`: devuelve el objeto situado en la posición especificada.
   - `indexOf(Multimedia m)`: devuelve la posición del objeto que se introduce por parámetro, si no se encuentra, devolverá -1.
   - __str__ devuelve la información de los objetos que están en la lista.

   Escriba una aplicación para:

   a) Crear un objeto de tipo `ListaMultimedia` de tamaño máximo 10.
   b) Crear tres películas y se añadan a la lista.
   c) Mostrar la lista por pantalla
   d) Crear un objeto de tipo `Película` introduciendo el título y el autor de una de las películas de la lista, para el resto de los argumentos se utilizan valores no significativos.
   e) Buscar la posición de este objeto en la lista.
   f) Obtener el objeto que está en esa posición y lo muestre por pantalla junto con la posición en la que se encuentra.

## OPCIONAL

2. Usando las clases `GeometricObject`, `Circle` y `Rectangle`, realizar lo siguiente:

   a) Implemente la clase `GeometricObject`. El inicializador `__init__` debe inicializar el atributo `color` con "`green`" y el atributo `filled` con `True`.

   b) Implemente la clase `Circle`. El constructor `__init__` debe recibir solamente un solo parámetro - el radio, y debe invocar a `__init__` del padre para inicializar los otros atributos. El método `printCircle` debe invocar a `__str__` del padre a fin de mostrar los atributos `color` y `filled`, asimismo, debe mostrar su atributo `radius`.

   c) Implemente la clase `Rectangle`. El inicializador `__init__` debe recibir dos parámetros – ancho y altura, y debe invocar a `__init__` del padre para inicializar los otros parámetros.

d) Implemente un programa para probar el mecanismo de herencia de las tres clases descritas.
   - Crea un objeto Circle con radio 1.5 de la siguiente manera: `circle=Circle(1.5)`
   - Ejecute las instrucciones: `circle.printCircle()` y `print(circle)`, ¿los atributos de qué objeto imprime esta última instrucción, la del padre o del hijo?.
   - Pruebe el funcionamiento de los métodos de Circle.
   - ¿Cómo imprimir solamente el atributo `color` de Circle?.
   - Crea un objeto Rectangle y pruebe sus métodos.
   - Modifique el color del objeto Rectangle.

3. *Sobreescritura de métodos*. El método __str__ de `GeometricObject` puede ser modificado en las clases hijo a fin de que muestre sus atributos de manera más precisa. Agregue el método __str__ a las clases Circle y Rectangle de tal manera que modifique __str__ del padre.

4. ¿Qué problema surge cuando se ejecuta el siguiente programa? ¿cómo resolverlo?

```
class A:
    def __init__(self, i = 0):
        self.i = i

class B(A):
    def __init__(self, j = 0):
        self.j = j

def main():
    b = B()
    print(b.i)
    print(b.j)

main() # Call the main function
```

5. ¿Qué imprime el siguiente programa?. Fíjese que `m1()` se encuentra en ambas clases.

```
class A:
    def __init__(self, i = 0):
        self.i = i
    def m1(self):
        self.i += 1

class B(A):
    def __init__(self, j = 0):
        super().__init__(3)
        self.j = j
    def m1(self):
        self.i += 1
def main():
    b = B()
    b.m1()
    print(b.i)
    print(b.j)

main() # Call the main function
```

Guarde todos vuestros programas en una carpeta con el nombre su **Apellido** paterno seguido de vuestro **DNI**, luego comprima esta carpeta. Envíe este archivo a Katherine Navarro katherine.navarro@upch.pe especificando como asunto **Lab7.2**.